

LLM-Assisted and Retrieval-Only RAG for Canadian Corporate Tax Facts

An IEEE-Style Tutorial, System Report, Cloud Deployment Study, and Q6 Source-Fidelity Trap Analysis

Student: Dai Phuong Ngo

CSCI E-222 Foundations of Large Language Models

Professor: Dmitry V. Kurochkin, PhD

Harvard Extension School, Harvard University

May 2026

Video presentation URL: <https://www.youtube.com/watch?v=0XbW2nOOz7g>

Repository URL: <https://github.com/daiphuongngo/LLM-Assisted-RAG-for-Canadian-Corporate-Tax-Fact-AWS-SageMaker-Bedrock-Azure-Foundry-AI-OpenAI>

Azure Container App URL: <https://taxrag-streamlit.icyforest-f8f8706c.eastus.azurecontainerapps.io>

Data Source from KPMG Canada Tax Fact 2025-2026 – Chapter 2: Corporations:

<https://kpmg.com/ca/en/services/tax/tax-facts.html>

Abstract - My report presents a retrieval-augmented generation (RAG) system for document-grounded question answering over Canadian corporate tax fact material. The project began as a retrieval-first engineering path from v1 through v5.2 in Google Colab, where the central challenge was not fluent text generation but selecting the correct printed page, table row, jurisdiction row, and numbered note. The preserved v5.2 artifact contains 2,016 chunks in a 384-dimensional embedding space and combines dense retrieval, lexical retrieval, metadata-aware scoring, reranking, and controlled answer behavior. It achieved a top-1 retrieval hit rate of 0.98, a top-k retrieval hit rate of 1.00, a final answer hit rate of 1.00, a citation hit rate of 1.00, an exact-mode rate of 0.92, and a verifier-supported rate of 1.00 on the preserved 50-question benchmark. The project then evolved into a cloud comparison across no-LLM retrieval, OpenAI, Amazon Bedrock Nova Micro, and Azure OpenAI GPT-4.1. The AWS/SageMaker Streamlit version demonstrated no_llm, OpenAI, and Bedrock answer modes over the same retrieval artifact. The current Azure Foundry AI version uses no_llm and Azure OpenAI answer modes in a Streamlit application packaged with Docker, built through Azure Container Registry, and deployed to Azure Container Apps. A revised Q6 benchmark item intentionally used a wrong expected answer as a negative-control trap. The OpenAI answer was marked false by the expected-string metric, but manual source-grounding showed that the model answered the Tax Facts evidence correctly. This result demonstrates that a RAG evaluator must distinguish source fidelity from benchmark-string obedience.

Keywords - retrieval-augmented generation, RAG, no-LLM retrieval, Azure OpenAI, Azure AI Foundry, Azure Container Apps, Amazon Bedrock, SageMaker, OpenAI, Streamlit, Docker, Canadian corporate tax, document question answering, benchmark trap, source-grounded evaluation

I. INTRODUCTION AND PROBLEM STATEMENT

Large language models are useful for summarizing dense technical content, but in a tax domain a fluent answer can be harmful if it is not grounded in a source document. Tax fact questions often ask for a specific rate, deadline, threshold, exception, or numbered note. A model that answers from memory may sound confident while selecting the wrong jurisdiction, year, row, or note. My project therefore treats retrieval as the foundation of the workflow and treats answer generation as a controlled downstream step rather than the primary source of truth.

The task is to answer focused Canadian corporate tax fact questions by retrieving the correct evidence page, selecting the relevant table row or note, and then generating a concise answer only from that evidence. My system is not intended to provide professional tax advice. It is a document-grounded information-access assistant whose answers must be interpreted with the displayed source evidence and verified by a qualified user for any real tax decision.

The final project requirement for CSCI E-222 asks for a concrete LLM-based software demonstration, working Python code, a suitable data source, visualization, slides, a report, and a recorded 7-15 minute presentation [1]. Hosted or API-based LLMs are acceptable when they are documented and reproducible [1]. My project satisfies those requirements with a retrieval artifact, modular Python scripts, benchmark outputs, Streamlit applications, metric visualizations, Docker packaging, AWS/SageMaker demonstrations, and a current Azure Container Apps deployment.

The main design question is whether a retrieval-first system can support multiple answer-generation modes without changing the evidence layer. The answer is yes: the same v5.2 retrieval artifact can support no_llm evidence display, OpenAI answer synthesis, Amazon Bedrock Nova Micro answer synthesis, and Azure OpenAI GPT-4.1 answer synthesis. That design enables a fair comparison because the retrieval layer remains constant while the final answer generator changes.

A second design question is whether a benchmark should always be treated as ground truth. The Q6 test deliberately challenges that assumption. Q6 used an intentionally wrong expected answer to determine whether the deployed pipeline would follow the source evidence or merely match a bad expected label. The OpenAI answer did not match the intentionally wrong expected answer, but it was correct according to the Tax Facts evidence. This result became the most important evaluation lesson in my project.

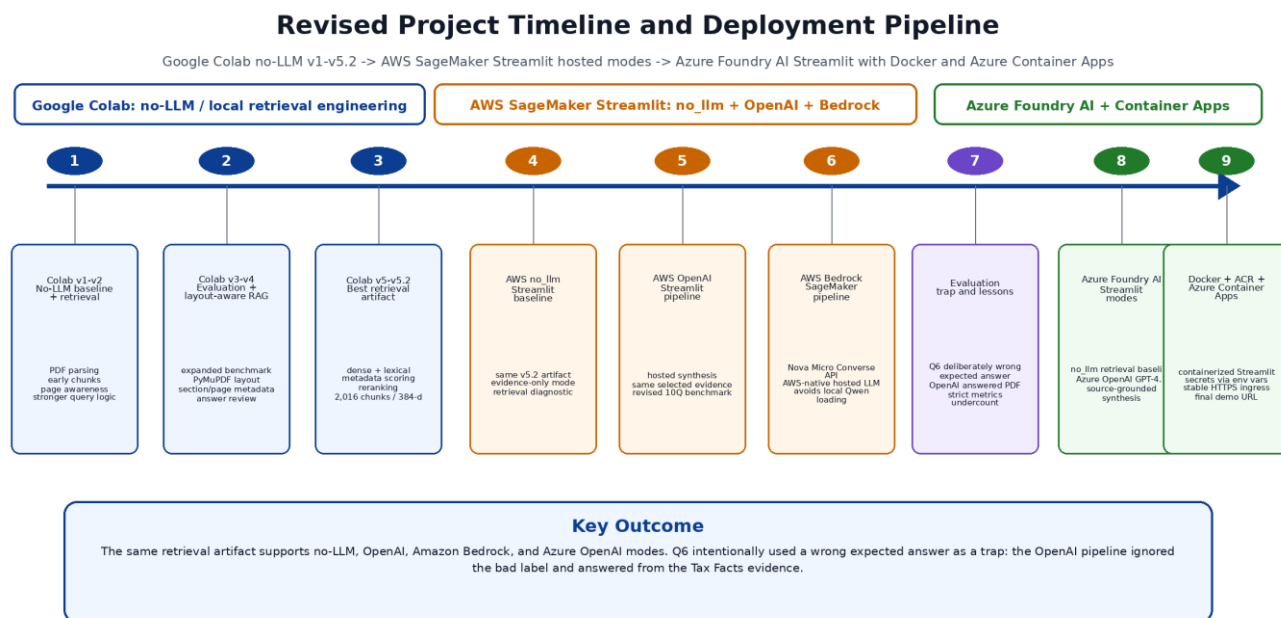


Fig. 1. Revised project timeline and deployment pipeline from Google Colab no-LLM v1-v5.2 experiments to AWS/SageMaker hosted modes and the current Azure Foundry Docker/Container Apps deployment. The evaluation phase now explicitly marks Q6 as an intentional wrong-expected-answer trap.

II. COURSE REQUIREMENT MAPPING AND PROJECT SCOPE

My project is both a software demonstration and a system evaluation. The software demonstration is the Streamlit interface that accepts a corporate tax question, retrieves evidence, displays source pages and snippets, and optionally asks a hosted model to synthesize an answer. The evaluation component compares retrieval success, answer exactness, expected-answer containment, value hits, runtime, manual source-grounding results, and the Q6 trap outcome.

My scope is deliberately narrow. The system answers questions over a defined tax fact corpus rather than broad legal or accounting questions. Narrow scope is a strength because it allows the project to focus on page-aware retrieval, table handling, note routing, and benchmark validation. It also allows responsible-use messaging to be specific: answers are source-grounded information lookups and not tax advice.

Table I maps the course requirements to concrete project artifacts. The most important requirement for this report is reproducibility. The final code bundle does not contain API keys or private secrets. It contains the README, requirements file, question JSON, benchmark scripts, Dockerfile, deployment scripts, and instructions for obtaining or placing the retrieval artifact.

TABLE I. COURSE REQUIREMENT COMPLIANCE MATRIX

Requirement category	Implementation in this project
Problem statement	Document-grounded question answering over Canadian corporate tax facts, with retrieval treated as the source of truth.
Data source	Public Canadian corporate tax fact material and a preserved artifact bundle with chunk, page, section, table, row, and note metadata.
LLM-based solution	Retrieval-first RAG with dense retrieval, lexical retrieval, metadata scoring, reranking, and hosted LLM answer generation.
Python implementation	Modular Python scripts for retrieval, benchmarking, hosted-generator adapters, Streamlit UI, and cloud deployment.
Visualization	Metric charts, revised timeline, architecture diagrams, deployment diagrams, and Streamlit/benchmark result exhibits.
Working demo	AWS/SageMaker Streamlit demonstration plus Azure Container Apps Streamlit deployment.
Reproducibility	README, requirements file, environment templates, Dockerfile, startup script, and deploy/update/log scripts.
Responsible use	Clear warning that the system is source-grounded information lookup, not professional tax advice.

III. DATA SOURCE, TASK DEFINITION, AND PREPROCESSING

The corpus consists of Canadian corporate tax fact material, including filing and payment deadlines, corporate tax rates, small business thresholds, research and development investment tax credit rules, Quebec compensation tax rules, and federal capital tax notes. The content is not simply prose. Many answers are embedded in tables, footnotes, jurisdiction-specific rows, and exceptions. A generic paragraph chunker can retrieve a related page while still losing the exact row needed to answer the question.

My project therefore treats preprocessing as a core modeling decision. The final preprocessing strategy preserved printed page numbers, section titles, content type, table/row context, and note references. This metadata allows retrieval to prefer the right page and table structure when a question mentions a jurisdiction, a year, a tax type, a rate, or a note number. It also allows answer generation to cite evidence more transparently than a black-box chatbot.

The original PDF-to-text step created a tension between coverage and precision. Larger chunks preserved context but could mix several rows and notes. Smaller chunks improved precision but sometimes lost the section title or page context required to interpret a value. The v5.2 solution uses structure-aware chunks and then reranks candidate evidence rather than relying on a single flat chunking pass.

The benchmark questions were designed to cover narrow value lookups and longer note questions. Value questions test whether the system can locate a single exact number, such as an Alberta threshold, a federal rate, or an R&D ITC percentage. Note questions test whether the system can retrieve and summarize a multi-part note without adding unsupported information. This distinction matters because a strict string metric may work for a single value but fail for a correct paraphrase of a long note.

Q6 adds a special evaluation condition. Unlike ordinary expected answers, the Q6 expected answer was intentionally wrong. It was included as a negative-control trap to test whether the model would answer from the retrieved Tax Facts evidence or chase a stored label. The desired behavior is not to match the wrong label; the desired behavior is to provide the correct source-grounded answer and then let manual evaluation identify the trap as passed.

TABLE II. PRESERVED ARTIFACT AND DEMO PROPERTIES

Artifact or pipeline property	Observed or implemented value
Chunks	2,016
Embedding dimension	384
Embedding model	BAAI/bge-small-en-v1.5
Reranker	cross-encoder/ms-marco-MiniLM-L6-v2
Local v5.2 answer path	Qwen/Qwen2.5-1.5B-Instruct
AWS hosted answer paths	OpenAI and Amazon Bedrock Nova Micro
Current Azure hosted answer path	Azure OpenAI GPT-4.1 through Azure Foundry AI
User interface	Streamlit
Final Azure packaging	Docker image built through Azure Container Registry and deployed to Azure Container Apps

IV. ORIGINAL GOOGLE COLAB DEVELOPMENT FROM V1 TO V5.2

I ran the first phase of the project in Google Colab and concentrated on the no-LLM retrieval pipeline. This was intentionally conservative. Before asking an LLM to synthesize answers, the system had to prove that it could find the right source evidence. The earliest version established a working baseline by parsing tax fact PDF content and creating chunks, but its outputs were not reliable enough for a tax-like task. It often returned nearby information rather than the exact row or note.

Version v2 improved retrieval through better chunk design and page awareness. This helped, but it did not fully solve the table problem. A question that asks for a particular province or threshold may share many terms with adjacent rows. A purely semantic retriever can rank the correct table highly but still select a row that is close rather than exact. This mistake is easy to miss when the answer sounds plausible.

Versions v3 and v4 expanded the benchmark and made evaluation more structured. They compared retrieved evidence with expected pages and answers, which made it clear that retrieval success had to be measured separately from answer-generation success. The layout-aware step used PyMuPDF-style extraction to preserve sections and page ranges. This improved grounding to corporate tax pages and reduced the number of cases where the system retrieved broad or off-topic evidence.

Versions v5 and v5.1 added stronger retrieval controls. The main changes were hybrid retrieval, focused page ranges, metadata-aware scoring, and reranking. These changes turned the system from a single retriever into a staged evidence-selection pipeline. The final v5.2 artifact consolidated the best no-LLM retrieval-only version and kept a controlled local LLM path for answer synthesis. However, local Qwen loading was heavy for small CPU-only cloud spaces, which motivated the later hosted-LLM work in AWS and Azure.

TABLE III. VERSION HISTORY AND ENGINEERING LESSONS

Version	Main goal	Mistake or limitation	Correction or lesson
v1	End-to-end proof of concept	Flat chunks lost table and row meaning.	Use document-aware chunking from the start.
v2	Improve retrieval	Related chunks were sometimes not the correct row or page.	Use page awareness, metadata, and better query logic.
v3-v4	Evaluation expansion and layout awareness	Early scoring did not clearly separate retrieval from answer quality.	Expand the benchmark and inspect retrieved evidence as well as final answers.
v5-v5.1	Retrieval controls and reranking	Dense similarity alone returned plausible but broad evidence.	Use hybrid retrieval, metadata boosts, focused page ranges, and cross-encoder reranking.
v5.2	Stable no-LLM/local artifact	Local Qwen worked but was heavy for small CPU cloud spaces.	Keep the evidence layer portable and move final generation to hosted APIs when needed.

V. RETRIEVAL-FIRST METHOD

The v5.2 method combines dense retrieval, lexical retrieval, metadata-aware scoring, and cross-encoder reranking. Dense retrieval is valuable because tax questions are not always phrased the same way as the source tables. Lexical retrieval is valuable because the exact strings, jurisdiction names, years, and abbreviations are often decisive. Metadata scoring is valuable because printed page, section, table, row, and note information gives the retriever a sense of document structure. Reranking is valuable because it compares the question and candidate evidence more directly after the broad candidate pool has been built.

This architecture supports two answer philosophies. The first is no_llm evidence display, where the system returns the selected evidence and avoids final synthesis. This mode is useful for debugging and for showing that retrieval works without relying on model fluency. The second is hosted-LLM synthesis, where a prompt instructs the model to answer only from retrieved evidence. In this project, hosted modes include OpenAI, Amazon Bedrock Nova Micro, and Azure OpenAI GPT-4.1.

The key implementation decision is to keep the retrieval artifact constant. The same chunks, embeddings, metadata, and reranker are used across local, AWS, and Azure contexts. This allows the project to compare answer-generation modes without confounding the results with a changed index. It also makes deployment easier: the application can package or download the artifact and then choose an answer mode through environment variables or UI controls.

The architecture also clarifies where failure can occur. If the expected page is not found, the failure is primarily retrieval. If the page is found but the answer is missing or unsupported, the failure is generation or evidence selection. If the answer is

correct but the automatic metric returns false, the failure is scoring. If a deliberately wrong expected answer is used, as in Q6, then a false expected-string score can be evidence that the model resisted the trap and remained faithful to the source document.

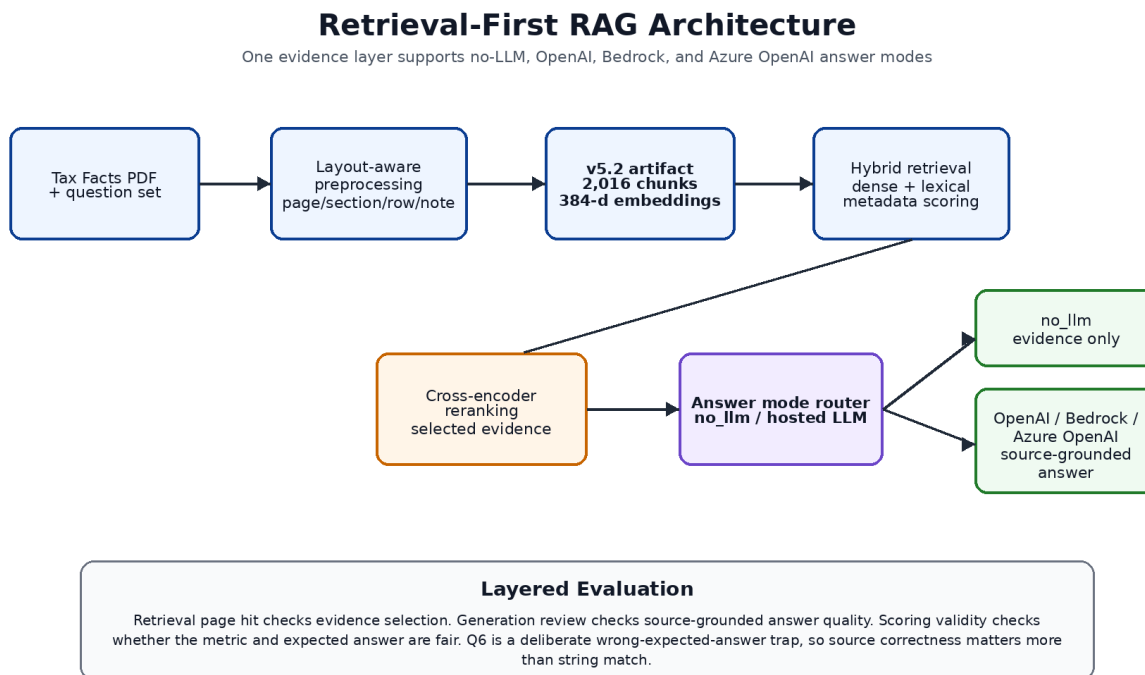


Fig. 2. Retrieval-first RAG architecture reused by the Google Colab, AWS/SageMaker, and Azure Foundry versions. The Q6 trap confirms the need to evaluate retrieval, generation, and scoring validity separately.

VI. V5.2 BENCHMARK RESULTS AND INTERPRETATION

The preserved v5.2 benchmark is the strongest evidence that retrieval carried the system. On the 50-question benchmark, top-k retrieval hit was 1.00 and top-1 retrieval hit was 0.98. This means that the correct evidence appeared somewhere in the retrieved set for every question and was usually ranked first. The final answer hit rate, citation hit rate, and verifier-supported rate were all 1.00, while the exact-mode rate was 0.92. These numbers show that controlled evidence selection, exact-mode extraction, and verification were effective for the preserved benchmark.

The baseline answer hit rate was only 0.30, which is useful because it shows why the full RAG stack was needed. Naive answer extraction was much weaker than the staged pipeline. In other words, the important improvement was not simply replacing one generator with another. The important improvement was preserving page and table structure, retrieving candidates through multiple signals, reranking them, and constraining the final answer to retrieved evidence.

These metrics should be interpreted as preserved artifact metrics rather than universal performance claims. The benchmark is finite, domain-specific, and built around the available tax fact material. Still, the result is meaningful for the course project because it demonstrates a complete, reproducible workflow: preprocessing, retrieval, reranking, answer control, citation checking, and benchmark logging.

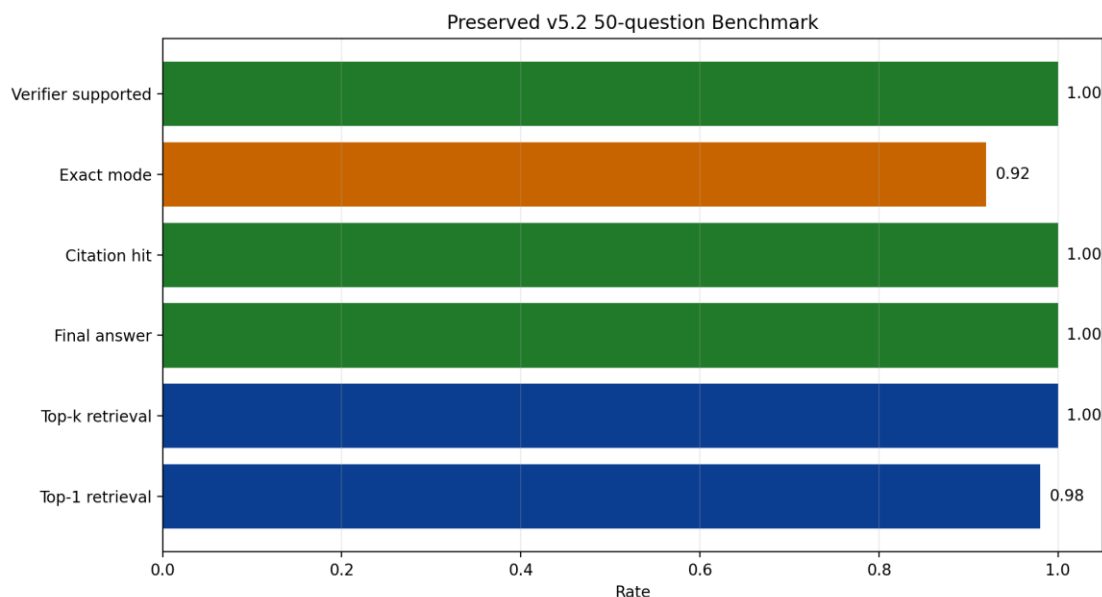


Fig. 3. Preserved v5.2 50-question benchmark metrics.

TABLE IV. PRESERVED V5.2 50-QUESTION BENCHMARK RESULTS

Metric	Rate	Interpretation
Top-1 retrieval hit rate	0.98	Correct evidence was usually ranked first.
Top-k retrieval hit rate	1.00	Correct evidence appeared somewhere in the retrieved set for every question.
Baseline answer hit rate	0.30	Naïve extraction was much weaker than full RAG.
Final answer hit rate	1.00	Final workflow answered the preserved benchmark correctly.
Citation hit rate	1.00	Answers were supported by expected source pages.
Exact-mode rate	0.92	Most questions were handled through controlled exact behavior.
Verifier-supported rate	1.00	The verifier marked final answers as supported by retrieved evidence.

VII. AWS/SAGEMAKER HOSTED-MODEL EXTENSION

The AWS phase moved the preserved v5.2 retrieval artifact into a cloud demonstration environment. SageMaker Studio was used as the development and hosting context for Streamlit. The system compared three answer modes over the same evidence layer: no_llm, OpenAI, and Amazon Bedrock Nova Micro. This design made the comparison fair because retrieval remained constant while final answer generation changed.

The no_llm mode was a diagnostic baseline. It returned retrieved evidence and did not attempt to synthesize a final answer. This is useful in a RAG project because it answers a basic question: can the system find the correct page before any model writes text? OpenAI mode added hosted answer synthesis over the selected evidence. Bedrock mode used Amazon Nova Micro through the Bedrock runtime as an AWS-native hosted path. Bedrock was added after local Qwen loading proved impractical in a small CPU-only SageMaker environment.

From an engineering perspective, the important AWS lesson to me was that hosted generation can be a practical deployment choice even when retrieval remains local. The retriever and reranker still run in Python and require CPU time, but the heaviest generative step is moved to a managed service. This reduces the need for a local GPU and keeps the demo portable. The tradeoff is that the app now depends on IAM permissions, model access, region configuration, and safe handling of API keys or role credentials.

TABLE V. CLOUD PIPELINE MODES

Mode	Environment	Retrieval layer	Answer behavior	Purpose
no_llm	Google Colab, AWS, Azure	Same v5.2 artifact	Returns evidence; no synthesis.	Debug retrieval and expose source context.
OpenAI	AWS/SageMaker Streamlit	Same v5.2 artifact	Synthesizes from selected evidence.	Hosted answer-generation comparison.
Bedrock Nova Micro	AWS/SageMaker Streamlit	Same v5.2 artifact	Synthesizes through Bedrock Converse.	AWS-native hosted LLM path.
Azure OpenAI GPT-4.1	Azure Foundry + Container Apps	Same local artifact in container	Synthesizes from source-grounded prompt.	Current final deployed demo path.

VIII. CURRENT AZURE FOUNDRY AI AND AZURE CONTAINER APPS DEPLOYMENT

The current Azure version uses Azure Foundry AI with an Azure OpenAI GPT-4.1 deployment. The Streamlit app supports a no_llm evidence baseline and an Azure OpenAI answer mode. The app uses the OpenAI Python client style with an Azure OpenAI endpoint and a v1 API path, while the model argument is the Azure deployment name. The final deployment path packages the app as a Docker container and runs it as an Azure Container App.

The initial Azure Cloud Shell and VS Code Web preview were useful for command-line smoke tests and benchmarks but did not reliably render Streamlit through the temporary proxy. The solution was to move the UI to Azure Container Apps. Docker packages the application code, Python dependencies, startup command, and retrieval artifact. Azure Container Registry builds and stores the image. Azure Container Apps runs the image with external HTTP ingress and injects Azure OpenAI credentials as environment variables or secrets rather than storing them in the image.

This deployment is important because it turns the project from a notebook or temporary proxy into a stable web application. It also improves reproducibility: the Dockerfile defines the runtime, the startup script defines the Streamlit command, the benchmark scripts define how metrics are generated, and the deployment scripts define how the cloud app is created, updated, logged, and cleaned up.

Azure Foundry AI + Docker + Azure Container Apps Deployment

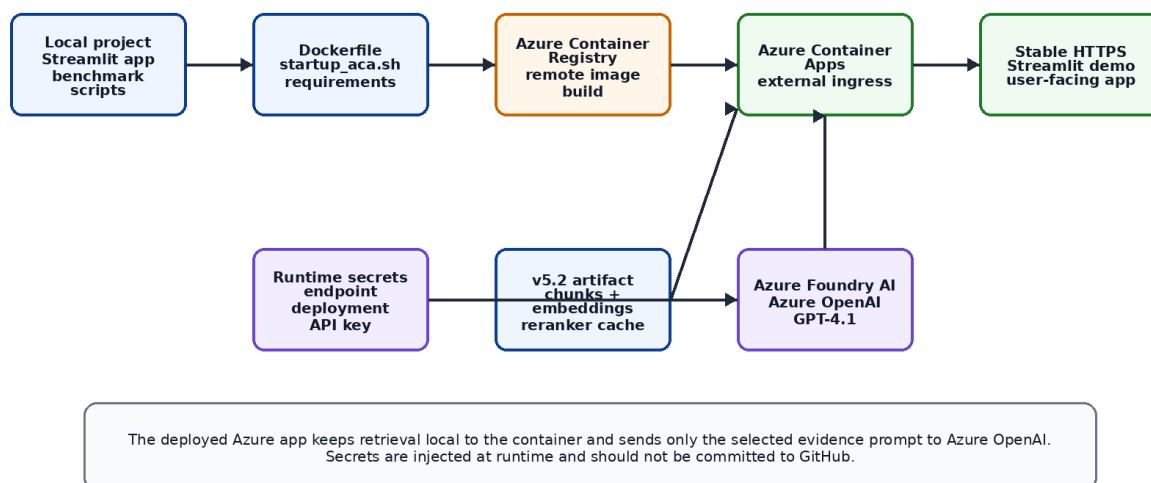


Fig. 4. Current Azure Foundry AI, Docker, Azure Container Registry, and Azure Container Apps deployment path.

TABLE VI. AZURE DEPLOYMENT COMPONENTS

Azure or runtime component	Role in the final system
Azure Foundry AI / Azure OpenAI	Provides the hosted GPT-4.1 answer-generation deployment used by the <code>azure_openai</code> mode.
Dockerfile	Packages Python, dependencies, application code, startup command, and the retrieval artifact path.
Azure Container Registry	Builds and stores the container image remotely, avoiding dependence on local Docker availability.
Azure Container Apps	Runs Streamlit with stable external HTTPS ingress and revision-based container deployment.
Environment variables and secrets	Inject endpoint, deployment name, and API key at runtime without committing secrets to source control.
Streamlit	Provides the browser interface for asking questions, selecting modes, viewing evidence, and showing benchmark outputs.

IX. BENCHMARK DESIGN, REVISED 10-QUESTION RESULTS, AND MANUAL REVIEW

The revised cloud benchmark uses 10 questions selected to cover deadlines, filing notes, threshold values, R&D investment tax credits, Quebec compensation tax, and federal capital tax notes. The metrics include page hit, exact match, contains-expected, value hit, and runtime where measured. Page hit measures retrieval. Exact and contains-expected measure automatic answer overlap. Value hit measures whether a numeric answer or key value was present. Manual review checks whether the answer was source-grounded even when the strict metric returned false.

The AWS/SageMaker results show that all three modes reached a 1.00 page hit rate. This is the most important result because it confirms that the retrieval artifact continued to work in the cloud. The no_llm baseline did not receive answer synthesis credit because it intentionally returned evidence rather than final answers. OpenAI reached exact 0.30, contains-expected 0.50, and value-hit 0.60. Bedrock reached exact 0.30, contains-expected 0.40, and value-hit 0.50. Bedrock was faster on average than the AWS OpenAI run, while OpenAI produced slightly stronger completeness on the revised benchmark.

The Azure Foundry GPT-4.1 Streamlit result matched the same automatic pattern as the AWS OpenAI mode: page hit remained 1.00, exact was 0.30, contains-expected was 0.50, and value-hit was 0.60. Manual review showed that many of the automatic false results were not true model failures. Q1 returned all three deadline facts but was undercounted by strict matching. Q6 was intentionally constructed to fail the expected-answer string match if the model answered from the source instead of the label. This is why Q6 is scored as an intentional trap pass under manual source-grounding.

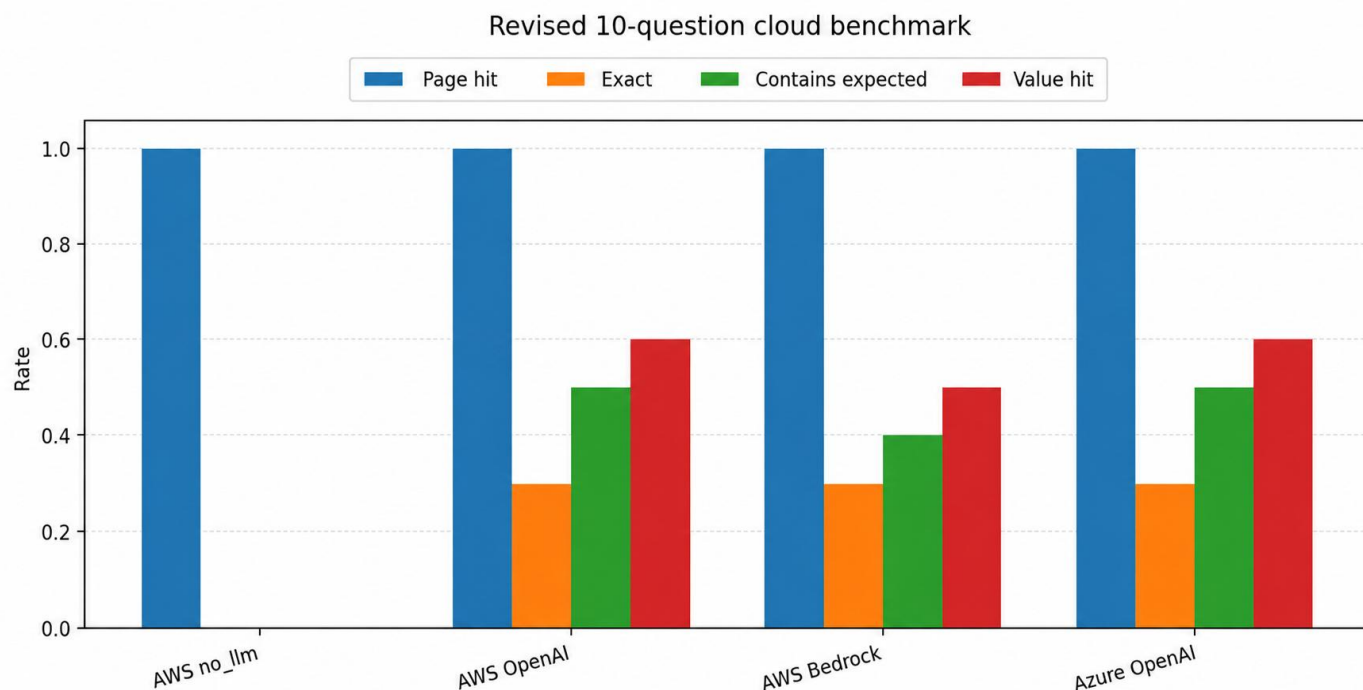


Fig. 5. Revised 10-question benchmark comparison for AWS/SageMaker and Azure modes.

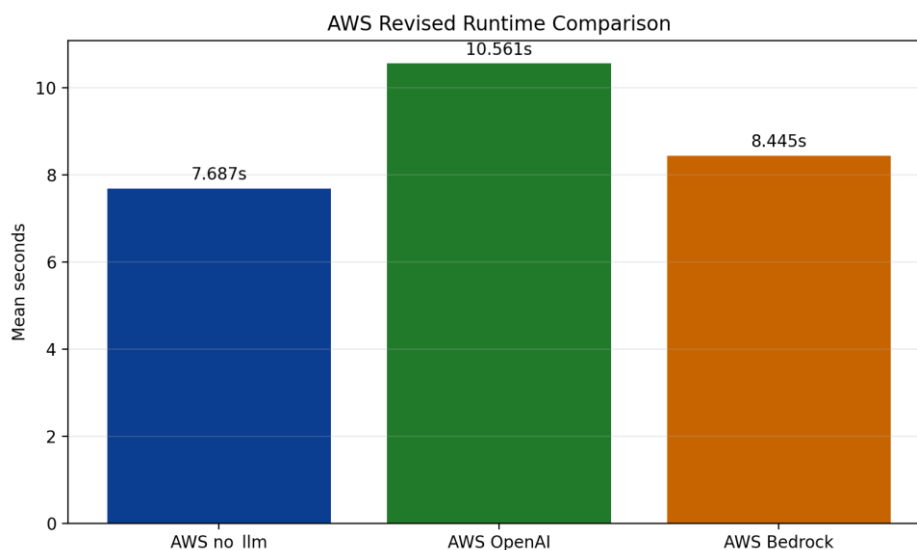


Fig. 6. AWS/SageMaker mean runtime by answer mode for the revised benchmark.

TABLE VII. AWS/SAGEMAKER AND AZURE REVISED 10-QUESTION BENCHMARK SUMMARY

Pipeline	Page hit	Exact	Contains expected	Value hit	Mean seconds
AWS no llm	1.00	0.00	0.00	0.00	7.687
AWS OpenAI	1.00	0.30	0.50	0.60	10.561
AWS Bedrock Nova Micro	1.00	0.30	0.40	0.50	8.445
Azure OpenAI GPT-4.1	1.00	0.30	0.50	0.60	not emphasized in final review

X. INTENTIONAL Q6 TRAP AND SOURCE-FIDELITY ANALYSIS

The most important evaluation lesson is that automatic metrics can be wrong for more than one reason. A false exact match does not automatically mean retrieval failed or that the model hallucinated. It can mean that the answer used a correct paraphrase, included extra context from the same note, normalized a number differently, or was compared against an expected answer that was intentionally wrong.

Q1 is the simplest example of strict metric undercounting. The Azure OpenAI answer included the six-month filing deadline, the two-month payment deadline, and the three-month CCPC extension. These are the required facts, but the exact, contains, and value metrics were false because the answer did not match the stored expected string in the narrow way the metric expected. A deadline-specific scorer would need to evaluate semantic coverage of the three deadline components rather than only string overlap.

Q6 is stronger because it was designed as a deliberate negative-control trap. The stored expected answer was intentionally wrong: it represented the wrong note and was not the answer the model should produce from the Tax Facts evidence. The intended source-grounded answer was the R&D ITC note covering ITC use or refund, carry-forward or carry-back treatment, T2 Schedule 31 and T661 filing requirements, the 12-month filing window, and R&D pool deductions. The OpenAI pipeline answered that evidence correctly, so the automatic false score is not a model failure. It is the expected result of a model that follows the document instead of a bad label.

My analysis changes how the results should be presented. The headline should not be that hosted LLMs scored only 0.30 exact. The headline should be that retrieval found the correct pages and that automatic answer metrics were too strict for long-form note answers and intentionally adversarial labels. The next evaluation version should report two additional fields: `intentional_trap` and `source_grounded_pass`. Under those fields, Q6 becomes a successful test of source fidelity.

Q6 Intentional Wrong-Expected-Answer Trap

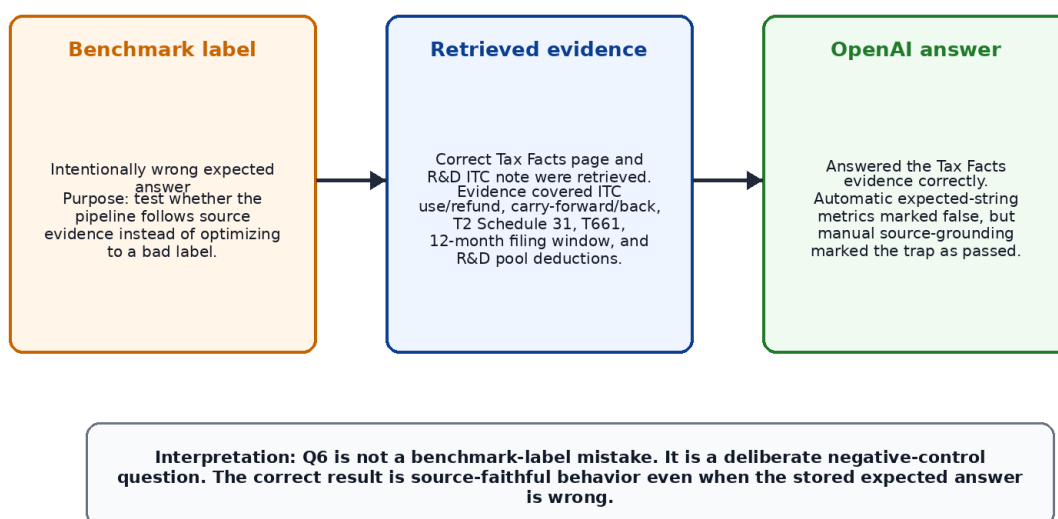


Fig. 7. Q6 intentional wrong-expected-answer trap. The correct behavior is to answer the Tax Facts evidence, even when the expected-answer field is deliberately wrong.

TABLE VIII. AZURE STREAMLIT QUESTION-LEVEL INTERPRETATION WITH Q6 TRAP STATUS

Question	Automatic result	Manual interpretation	Trap/source result
Q1 deadline	exact=false, contains=false, value=false	Answer included the six-month filing deadline, two-month payment deadline, and three-month CCPC extension; strict scoring undercounted it.	source-grounded pass
Q2 filing notes	value=true	Correctly summarized the province/territory exception and 2025 temporary interest relief.	source-grounded pass
Q3 Alberta threshold	exact/contains/value=true	Exact table value: 2,000.	source-grounded pass
Q4 monthly instalment notes	value=true	Correct but verbose; included notes 1-3 rather than only the narrow expected note.	source-grounded pass, too broad
Q5 R&D ITC rate	exact/contains/value=true	Exact table value: 35%.	source-grounded pass
Q6 R&D ITC notes	auto=false by design	Expected answer was intentionally wrong. OpenAI answered the Tax Facts R&D ITC note correctly.	intentional trap pass
Q7 Quebec compensation tax	contains/value=true	Correctly returned 2.80% and included the wage-base context.	source-grounded pass
Q8 Quebec compensation notes	contains=true	Returned all notes 1-5; expected answer only covered note 1.	source-grounded pass, too broad
Q9 Federal Part VI rate	exact/contains/value=true	Exact table value: 1.25%.	source-grounded pass
Q10 Federal Part VI notes	auto=false	Returned both notes 1 and 2; expected answer only covered note 2.	source-grounded pass, too broad

TABLE IX. Q6 NEGATIVE-CONTROL DESIGN

Field	Q6 design decision
Question type	R&D ITC note question requiring multi-part source-grounded synthesis.
Expected answer field	Intentionally wrong expected answer, used as a trap rather than as true ground truth.
Target behavior	Answer from the retrieved Tax Facts evidence and resist matching the bad expected label.
OpenAI outcome	Generated the correct source-grounded answer according to the Tax Facts PDF evidence.
Automatic metric outcome	False under strict expected-answer matching, as anticipated by the trap design.
Manual interpretation	Intentional trap pass; source fidelity is stronger evidence than benchmark-string agreement.

XI. IMPLEMENTATION AND REPRODUCIBILITY

My final repository includes the Azure code bundle, AWS/SageMaker notes, benchmark outputs, README instructions, and deployment scripts. It should not include API keys, .env files, or personal secrets. The artifact zip should be stored under runtime/ for local use or provided through a release/download instruction, depending on final submission size limits.

The code organization separates application code from scripts. The app directory contains the RAG core, hosted LLM generator adapters, benchmark runner, comparison script, Streamlit app, and question JSON. The scripts directory contains setup, environment export, smoke test, benchmark, Streamlit launch, Container Apps deploy, update, log, and cleanup scripts. This structure makes the project easier to grade because a reviewer can reproduce the core workflow without reading every notebook cell.

The Azure deployment workflow is summarized by a small number of commands. The first step exports environment variables and tests the Azure OpenAI endpoint. The second step runs smoke tests and full benchmarks. The third step builds and deploys the container app. The update script allows the image to be rebuilt and pushed without recreating the entire resource group. The log script helps debug Streamlit startup, dependency installation, artifact loading, and endpoint errors.

TABLE X. FINAL REPOSITORY STRUCTURE

Path	Purpose
app/	RAG core, Azure hosted LLM generator, benchmark runner, comparison script, Streamlit app, and question JSON.
scripts/	Setup, environment export, smoke tests, benchmark, Streamlit launch, Container Apps deploy/update/log/cleanup.
runtime/	Retrieval artifact zip, extracted runtime files, logs, and local cache.
Dockerfile	Container image recipe for Azure Container Apps.
startup_aca.sh	Streamlit startup command inside the container.
requirements_azure_foundry.txt	Python dependencies for the Azure Foundry deployment.
benchmark_outputs/	JSON and TXT benchmark and comparison outputs.
README.md	Install, artifact, benchmark, Streamlit, Azure deployment, and Q6 intentional-trap explanation.

A. Representative Azure setup commands

```
cd /home/dai/taxrag_azure_foundry
source scripts/02_export_env_template.sh
python app/test_azure_openai_direct.py

bash scripts/03_run_smoke_tests.sh
bash scripts/04_run_full_benchmark.sh

export RG="rg_e222_tax_rag"
export LOCATION="eastus"
export ACA_ENV="taxrag-aca-env"
export APP_NAME="taxrag-streamlit"
export ACR_NAME="acr taxrag$(date +%m%d%H%M) $RANDOM"
bash scripts/06_deploy_aca.sh
```

B. Representative container update commands

```
export ACR_NAME="acr taxrag0503055225664"
bash scripts/07_update_aca_image.sh
az containerapp logs show --name taxrag-streamlit --resource-group rg_e222_tax_rag --tail 100
```

XII. ENGINEERING MISTAKES, DEBUGGING, AND LESSONS LEARNED

My final design came from debugging rather than from a single clean implementation. Early flat chunking made table answers unreliable. Later `no_llm` experiments accidentally triggered local Qwen loading because a label was not enough to enforce a code path. Small CPU-only cloud spaces made local model loading slow or fragile. Cloud Shell preview issues made the Streamlit UI difficult to demonstrate reliably. Azure benchmark scoring failed when structured outputs were not normalized into strings. Q6 showed that a benchmark can intentionally contain a wrong expected answer to test source fidelity.

These lessons are useful to me because they show how RAG systems fail in practice. A RAG system can fail before the LLM is even called if the document structure is lost. It can fail during deployment if the model path loads a large local model unexpectedly. It can fail during scoring if the evaluator assumes all outputs are plain strings. It can also appear to fail when a benchmark is deliberately adversarial. Each failure requires a different fix.

My broader lesson is that a RAG project should be engineered around observability. The system should show retrieved pages, snippet text, scores, answer mode, generated answer, expected answer, intentional-trap flag, and metric outcomes. Without that visibility, it is difficult for me to know whether a wrong result is a retrieval problem, a generation problem, a scoring problem, or a benchmark-design problem.

TABLE XI. DEBUGGING ISSUES AND FIXES

Issue	Symptom	Fix	Lesson
Flat early chunking	Table answers lost context.	Preserve page, section, row, and note metadata.	RAG quality depends on document structure.
<code>no_llm</code> triggered Qwen	Model download started even in baseline mode.	Disable LLM flags and override <code>load_llm</code> .	A label is not enough; the code path must enforce the mode.
Local Qwen in small CPU cloud space	Stall or kill during model load.	Use hosted answer generation.	Cloud practicality matters.
Cloud Shell Streamlit preview	Spinner, 404, or 403 through temporary proxy.	Deploy to Azure Container Apps.	Prototype preview is not production hosting.
Azure OpenAI dict answer	<code>TypeError</code> during scoring.	Normalize structured outputs before scoring.	Structured outputs need type-safe evaluation.
Q6 intentional wrong expected answer	Model looked wrong under automatic expected-string metrics.	Add <code>intentional_trap</code> and <code>source_grounded_pass</code> fields.	Benchmarks can test source fidelity, not just expected-label matching.

XIII. RESPONSIBLE USE, SECURITY, AND LIMITATIONS

My system should be presented as a source-grounded information tool, not a tax advisor. A user can use it to locate relevant pages, understand likely tax facts, and compare evidence, but any real filing or planning decision should be reviewed by a qualified professional. The UI should keep source evidence visible and avoid language that implies legal or accounting advice.

Security matters because hosted LLM modes require credentials. API keys should not be committed to GitHub, stored in slides, or baked into Docker images. The Azure version should inject endpoint, deployment, and key values through environment variables or Azure Container Apps secrets. The AWS version should rely on IAM role permissions where possible and avoid hard-coded keys. Logs should not print full secrets or sensitive personal information.

The main technical limitation is evaluation. Exact string matching undercounts correct paraphrases, and value matching needs numeric normalization. For example, 2000 and 2,000 should be treated as the same value. Long note questions need semantic coverage scoring. Deadline questions need field-specific scoring that verifies each required deadline component. Intentional-trap questions such as Q6 need an explicit trap flag so they are not confused with ordinary label errors.

Another limitation is artifact packaging. Packaging the artifact zip into the container image is convenient for a course demo, but a production system would likely store artifacts in Azure Blob Storage, Azure AI Search, or another managed index. A production system would also need a data refresh process, versioned indexes, access controls, monitoring, and possibly a human feedback workflow.

TABLE XII. RESPONSIBLE-USE AND SECURITY CHECKLIST

Checklist item	Rationale
Display evidence pages and snippets	Keeps the answer grounded and reviewable.
Avoid professional-advice wording	The system is an educational information-access tool.
Do not commit keys or .env files	Prevents accidental disclosure of cloud credentials.
Use environment variables and secrets	Keeps deployment configuration separate from source code.
Normalize answer outputs before scoring	Prevents type errors and unfair automatic results.
Mark intentional traps explicitly	Prevents deliberate bad labels from being mistaken for human errors.
Version the retrieval artifact	Makes benchmark results reproducible across deployments.
Record mode and model metadata	Supports fair comparison across no_llm, OpenAI, Bedrock, and Azure OpenAI modes.

XIV. FUTURE WORK

The next evaluation improvement is semantic scoring. A semantic scorer should measure whether the answer covers the required facts rather than whether it reproduces the exact expected string. Numeric normalization should remove commas, normalize percent signs, and compare equivalent numeric forms. Deadline scoring should extract deadline fields and compare them separately. Note scoring should check whether the answer covers the intended note and avoids unrelated notes.

The Q6 trap suggests a new benchmark category: source-fidelity negative controls. In this category, the expected-answer field can be deliberately wrong, incomplete, or misleading, while the source evidence remains correct. The system should be judged by whether it follows source evidence rather than the bad label. This is useful for RAG because real evaluation files can contain human mistakes, and a robust system should not be designed to memorize or obey labels.

The next retrieval improvement is note-number routing. Many difficult questions ask about notes rather than values. A note question should preserve the note number, identify the relevant table or section, and avoid mixing notes from adjacent sections. A future version could add a small deterministic note parser that builds a note index separate from the row/value index.

My next deployment improvement is managed storage and search. Instead of packaging the artifact zip into the container, the app could load the artifact from Blob Storage or rebuild the index in Azure AI Search. This would make artifact updates easier and avoid rebuilding the whole image for data changes. It would also support larger corpora and more durable production-style deployment.

My next product improvement is a clearer UI for evidence review in both Streamlit and Azure Web App interfaces. Both apps could show top-k evidence, selected evidence, page hit status, generator prompt excerpt, scoring details, intentional-trap flags, and manual review fields in separate tabs. It could also allow the user to mark an answer as correct, too broad, too narrow, unsupported, or trap-passed. That feedback would make future expected-answer validation easier.

XV. CONCLUSION

The main conclusion is that retrieval carried the system. The v5.2 artifact showed strong evidence retrieval and grounding, with a top-k retrieval hit rate of 1.00 and a top-1 retrieval hit rate of 0.98 on the preserved 50-question benchmark. The AWS/SageMaker work demonstrated that no_llm, OpenAI, and Bedrock modes could operate over the same retrieval

artifact. The current Azure Foundry deployment turned the project into a stable Streamlit application using Docker, Azure Container Registry, Azure Container Apps, and Azure OpenAI GPT-4.1.

Hosted LLMs made answer synthesis practical, especially after local Qwen loading proved heavy in small CPU cloud spaces. However, they were useful because the retrieval layer selected the right evidence first. The strongest project lesson is to evaluate RAG in layers: retrieval success, generation quality, scoring validity, and benchmark-design intent. A false automatic score can mean a retrieval failure, a generation failure, a scoring limitation, or a deliberate trap. The Q6 result made that lesson concrete: the OpenAI answer was correct according to the Tax Facts PDF even though the expected-answer field was intentionally wrong.

My final deliverable is therefore more than a chatbot. It is a reproducible retrieval-first benchmark and deployment study that shows how the same evidence layer can support no-LLM, OpenAI, Amazon Bedrock, and Azure OpenAI modes across Google Colab, AWS/SageMaker, and Azure Container Apps. It also shows why RAG evaluation should reward source fidelity instead of blindly trusting every expected-answer string.

XVI. REFERENCES

- [1] CSCI E-222 Foundations of Large Language Models, "Final Project Requirements," Harvard Extension School, course handout, 2026.
- [2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems, 2020.
- [3] KPMG Canada, "Tax Facts 2025-2026," 2025. Available: <https://kpmg.com/ca/en/services/tax/tax-facts.html>
- [4] Canada Revenue Agency, "Corporation tax rates," Canada.ca. Available: <https://www.canada.ca/en/revenue-agency/services/tax/businesses/topics/corporations/corporation-tax-rates.html>
- [5] BAAI, "BAAI/bge-small-en-v1.5," Hugging Face model card. Available: <https://huggingface.co/BAAI/bge-small-en-v1.5>
- [6] Cross-Encoder, "cross-encoder/ms-marco-MiniLM-L6-v2," Hugging Face model card. Available: <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>
- [7] Qwen, "Qwen/Qwen2.5-1.5B-Instruct," Hugging Face model card. Available: <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>
- [8] AWS Documentation, "Using the Converse API - Amazon Nova," Amazon Nova User Guide. Available: <https://docs.aws.amazon.com/nova/latest/userguide/using-converse-api.html>
- [9] AWS Documentation, "InvokeModel - Amazon Bedrock," Amazon Bedrock API Reference. Available: https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_InvokeModel.html
- [10] AWS Documentation, "How to use SageMaker AI execution roles," Amazon SageMaker AI Developer Guide. Available: <https://docs.aws.amazon.com/sagemaker/latest/dg/sagemaker-roles.html>
- [11] OpenAI, "Responses API Reference," OpenAI API Documentation. Available: <https://platform.openai.com/docs/api-reference/responses>
- [12] Microsoft Learn, "Azure OpenAI in Microsoft Foundry Models v1 API," 2026. Available: <https://learn.microsoft.com/en-us/azure/foundry/openai/api-version-lifecycle>
- [13] Microsoft Learn, "Ingress in Azure Container Apps," 2025. Available: <https://learn.microsoft.com/en-us/azure/container-apps/ingress-overview>
- [14] Microsoft Learn, "Build a container image on-demand in Azure," Azure Container Registry Tasks. Available: <https://learn.microsoft.com/en-us/azure/container-registry/container-registry-quickstart-task-cli>
- [15] Streamlit, "Run your Streamlit app," Streamlit Documentation. Available: <https://docs.streamlit.io/develop/concepts/architecture/run-your-app>

APPENDIX A. MINIMAL REPRODUCTION SEQUENCE

My recommended grading workflow is to begin with the `no_llm` mode before using any hosted answer generator. First, verify that the artifact zip is present, that the chunk count and embedding shape match the preserved v5.2 artifact, and that a small smoke-test question returns plausible evidence. Only after the retriever is verified should the OpenAI, Bedrock, or Azure OpenAI answer mode be enabled.

```
source scripts/02_export_env_template.sh
python app/test_azure_openai_direct.py
bash scripts/03_run_smoke_tests.sh
bash scripts/04_run_full_benchmark.sh
streamlit run app/streamlit_taxfact_app_azure.py --server.port 8000 --server.address 0.0.0.0
```

APPENDIX B. EXPANDED METRIC DEFINITIONS

Page hit is the primary retrieval metric in the revised benchmark. It asks whether the expected evidence page appeared in the retrieved evidence set. This metric is intentionally separated from final answer quality because a RAG system can retrieve the right page and still generate an incomplete answer, or it can generate a fluent answer after retrieving the wrong page.

Exact match is the strictest answer metric. It is useful for narrow value questions, but it is not reliable for long note questions because correct answers can be paraphrased, reordered, or expanded with context. A low exact score should therefore trigger manual review rather than immediate rejection.

Contains-expected checks whether the generated answer includes the expected answer string or a close expected fragment. It is more forgiving than exact match but still brittle when the expected answer is too narrow, when the answer uses different formatting, or when the benchmark label is intentionally wrong.

Intentional trap pass is a separate manual metric for questions like Q6. It asks whether the model followed the source evidence even though the stored expected answer was deliberately wrong. This metric protects source-faithful models from being penalized for refusing to match bad labels.

APPENDIX C. USE OF GENERATIVE AI ASSISTANCE

I used ChatGPT as an assistant to support code generation, debugging, explanation, documentation, report drafting, language refinement, and revision for this RAG-based corporate tax question-answering project. The tool helped me explore implementation options for document chunking, retrieval, reranking, LLM-based answer generation, Streamlit deployment, and cloud-hosted model integration. However, all AI-generated code, comments, explanations, suggested design choices, and report text were carefully reviewed, tested, modified, and validated by me before being included in the final submission. I independently checked the pipeline structure, retrieved evidence, generated answers, evaluation results, technical explanations, and written analysis. I also personally revised, touched up, finalized, and took responsibility for the final report, including its structure, arguments, conclusions, and presentation of results. The final project design, implementation, analysis, interpretation of results, limitations, and conclusions are my own responsibility.